

LabelScan

De l'étiquette à la fiche
de traçabilité.

Capturer

Photo et code-barres

Structurer

OCR et extraction

Confirmer

Revue humaine

3

métiers

2

interfaces

1

parcours partagé

● Usages

● Architecture

● Données

● API

● Plateforme

Repères de lecture

LabelScan réunit une application de capture mobile, un back-office et un serveur de traitement. Une photo d'étiquette devient une proposition structurée, puis une fiche confirmée par l'opérateur. Cette documentation présente le produit, ses composants et les échanges qui les relient.

Partie	Contenu
01-06 · Expérience	Produit, parcours, profils, capture, revue et back-office
07-12 · Architecture	Composants, modules, traitement, séquence, états et modèle IA
13-15 · Données	Provenance, modèle relationnel et catalogue
16-18 · Interface API	Familles de routes, capture, revue et consultation
19-21 · Plateforme	Exécution, observabilité et qualité logicielle
22 · Références	Sources exécutables correspondant aux sections
G01-G10 · Guide du dépôt	Installation, lecture du code, tests, livraison et repères RNCP 5

Les quatre schémas sont des vues UML simplifiées : composants, séquence, états et classes de données. Leurs éléments sont éditables dans le [fichier Excalidraw](#). Les noms techniques correspondent aux identifiants du dépôt ; les flèches expriment un appel, une transition ou une association selon la légende.

Le document est une synthèse du code LabelScan. Les numéros de version cités décrivent les dépendances et contrats enregistrés dans le dépôt. Les références S01 à S16 permettent de retrouver les implémentations détaillées. L'[inventaire OpenAPI](#) complète les exemples HTTP.

Le [guide pédagogique](#) accompagne la prise en main du dépôt. L'édition PDF réunit la synthèse technique et ce guide en 34 pages.

01 - Le produit

LabelScan accompagne la réception de produits alimentaires. L'opérateur photographie l'étiquette fournisseur, consulte les informations proposées et confirme la fiche qui rejoint le catalogue du magasin. L'image, les extractions et la revue restent reliées à la même capture.

L'expérience articule trois fonctions : **collecter la preuve, structurer l'information, retrouver l'arrivage**. Elle couvre les métiers Poissonnerie, Boucherie et Charcuterie / Traiteur au sein d'une même plateforme.

Application mobile

Fonction principale

Capture, suivi du traitement, revue et consultation

Utilisateurs

Managers affectés à un portail métier

Back-office web

Fonction principale

Recherche d'arrivages, détail et administration

Utilisateurs

Managers, administrateurs et super-administrateurs

API

Fonction principale

Sessions, captures, extractions, fiches et annuaire

Utilisateurs

Applications clientes

Traitements asynchrones

Fonction principale

OCR, extraction, traçabilité et alertes

Utilisateurs

Services internes

Un **arrivage** est la vue consultable d'un lot enregistré à partir d'une capture revue. Le **portail métier** rattache ce travail à une organisation, un magasin et une profession. Ce contexte accompagne les données tout au long du traitement.

La fiche garde les champs du profil choisi côté serveur. Les champs qui l'autorisent peuvent recevoir la décision humaine NC. Les propositions machine et les décisions humaines portent des sources distinctes dans l'historique.

Références : S01, S02, S03, S04.

02 - Un parcours de bout en bout

Le parcours commence sur le terrain et se poursuit dans les outils de consultation. Plusieurs captures peuvent être en cours pendant que l'opérateur poursuit sa réception.

	ACTION	RÉSULTAT
1	Connexion Le manager ouvre sa session	Le serveur résout le magasin, le portail et le métier
2	Capture L'opérateur cadre l'étiquette et déclenche	Une image préparée et son contexte de code-barres sont disponibles
3	Envoi Le mobile soumet la capture	L'API retourne l'identifiant de l'ingestion acceptée
4	Extraction Les services lisent et structurent l'étiquette	Les propositions apparaissent avec leur état de validation
5	Revue L'opérateur renseigne les valeurs et les décisions d'absence autorisées	Une version humaine complète est enregistrée
6	Consultation Le catalogue est alimenté	L'arrivage peut être recherché et ouvert sur mobile ou web

Le mobile associe une carte à chaque scan local. Cette carte permet de suivre l'envoi et le traitement, puis d'ouvrir la revue. Les identifiants locaux et serveur relient les étapes sans demander à l'opérateur de les manipuler.

La reprise réseau conserve la clé d'idempotence de l'opération. Une répétition du même envoi est reconnue côté serveur. Une nouvelle photographie constitue, elle, une nouvelle capture.

La confirmation de revue enregistre le travail de l'opérateur. La publication catalogue s'effectue ensuite par les consommateurs de l'événement `review.finalized`.

Références : S02, S03, S05, S07.

03 - Trois profils métier

Le contrat actif est la version 2. Il partage douze champs et ajoute les champs propres à chaque profession. Le même profil structure l'extraction, la revue et l'affichage de la fiche.

Poissonnerie

16 champs

Nom scientifique, zone FAO, méthode de production, engin de pêche ou méthode d'élevage

Boucherie

21 champs

Espèce et catégorie animale, morceau, pays de naissance, élevage, abattage et découpe, agréments d'abattage et de découpe

Charcuterie / Traiteur

21 champs

Famille, fabricant, ingrédients, additifs, préparation, conditionnement, mode de conservation, utilisation et réchauffage

Les champs communs sont la désignation commerciale, le producteur, la marque revendeur, le lot, le pays d'origine, les dates de péremption et de conditionnement, la température de conservation, les allergènes, la marque sanitaire, le poids et le GTIN.

Le serveur définit les noms et types canoniques. Les clients présentent les libellés métier et les groupes de champs en français. Le code charcuterie_traiteur correspond à un seul profil combiné.

Les dates normalisées utilisent YYYY-MM-DD. Le poids accepte g et kg. Le GTIN couvre les longueurs 8, 12, 13 et 14 avec validation. La méthode de production distingue wild_caught et farmed.

L'ingestion conserve le code et la version du profil au moment de la capture. Le contrat historique v1 reste résoluble pour ses données ; il comprend le champ prix, retiré de la v2.

Références : S01, S02, S04.

04 - L'expérience mobile

L'application utilise Expo SDK 57, React Native 0.86 et React 19.2. Sa navigation principale comporte cinq écrans : connexion, articles, caméra, revue et détail d'article.

La caméra affiche un guide de cadrage, une commande de lampe et un déclencheur. Un verrou synchrone évite de lancer deux captures natives simultanément. Sur Android, la caméra suit le focus de l'écran et l'état actif de l'application.

La préparation remet la photo dans l'orientation attendue, applique la géométrie du guide, produit un résultat paysage et écrit un JPEG compressé. Le fichier est copié dans le stockage des captures en attente avant la création de l'opération d'envoi.

Fonction	Comportement
Codes-barres	Détection EAN, UPC, Code 128, Code 39 et QR ; contexte joint à la capture
Prises successives	Retour du déclencheur pendant que les envois précédents progressent
Suivi	Cartes de scans en cours, résultat disponible ou état d'erreur
Photographie	Affichage protégé, zoom, déplacement et rotation selon l'écran
Session	Restauration contrôlée avant ouverture du parcours authentifié

Les états réseau sont intégrés à l'interface. Au démarrage, les échecs temporaires de restauration affichent une attente de connexion et déclenchent des tentatives espacées. Pendant une session ouverte, la file d'envoi permet de conserver du travail à transmettre.

Le contexte de session comprend l'organisation, l'acteur, le portail et le métier. Les opérations locales et les clés de consultation utilisent cette identité complète.

Références : S02, S03, S12.

05 - La revue humaine

L'écran de revue rassemble la photographie et les champs du profil. Il suit le scan local et affiche les résultats disponibles : aperçu intermédiaire puis extraction finale.

Chaque champ reste visible. L'opérateur peut conserver une proposition, la corriger ou saisir NC pour les champs qui l'autorisent. La désignation, les dates de péremption et de conditionnement, l'origine, la marque sanitaire, le lot et la température exigent une valeur dans le formulaire. En poissonnerie, cette règle couvre aussi la méthode de production et la zone FAO ; pour l'élevage, la zone FAO reçoit NC en l'absence de saisie explicite non vide.

Élément	Rôle dans la revue
Photographie	Support visuel pour confronter la proposition à l'étiquette
Valeur proposée	Texte extrait ou issu du code-barres
État du champ	Présence, absence, ambiguïté ou état de normalisation
Décision humaine	Valeur renseignée ou NC explicite
Brouillon	Reprise locale de la saisie avant finalisation

Les contrôles du formulaire portent notamment sur les dates, GTIN, méthodes de production et champs complétés. Les suggestions d'allergènes en poissonnerie sont présentées à l'opérateur pour sélection.

La finalisation transmet l'ensemble exact des champs du profil. Le serveur vérifie la complétude, crée une nouvelle extraction dont les valeurs sont de source humaine, enregistre la rotation et place l'ingestion à l'état confirmed.

La run machine reste disponible dans l'historique. La run humaine porte une nouvelle tentative et un nouvel identifiant. L'événement review.finalized relie cette décision au traitement de traçabilité.

La fiche enregistrée est consultable depuis la liste et le détail catalogue. L'écran mobile propose également « Modifier la fiche » pour accéder aux corrections unitaires. Les interfaces affichent le profil complet.

Références : S02, S05, S07, S08.

06 · Le back-office

Le back-office est une application React servie par le backend. Son adresse suit la forme `/backoffice/o/{organization_slug}/`. Les écrans, filtres et libellés métier sont en français.

Rôle	Fonctions accessibles dans son périmètre
manager	Arrivages de son portail et compte personnel ; accès mobile
admin	Administration déléguée des magasins et portails attribués
super_admin	Gestion des magasins, administrateurs, managers et arrivages de son organisation

La recherche combine texte, période et filtres métier. Le serveur applique le périmètre autorisé avant recherche, tri, pagination et comptage. Le détail rassemble les informations du profil et la photographie correspondante.

Les définitions de présentation sont regroupées par profession dans le registre de portails web. Elles organisent les sections, les libellés et l'affichage des champs. L'annuaire relie les magasins, les portails actifs et les comptes affectés.

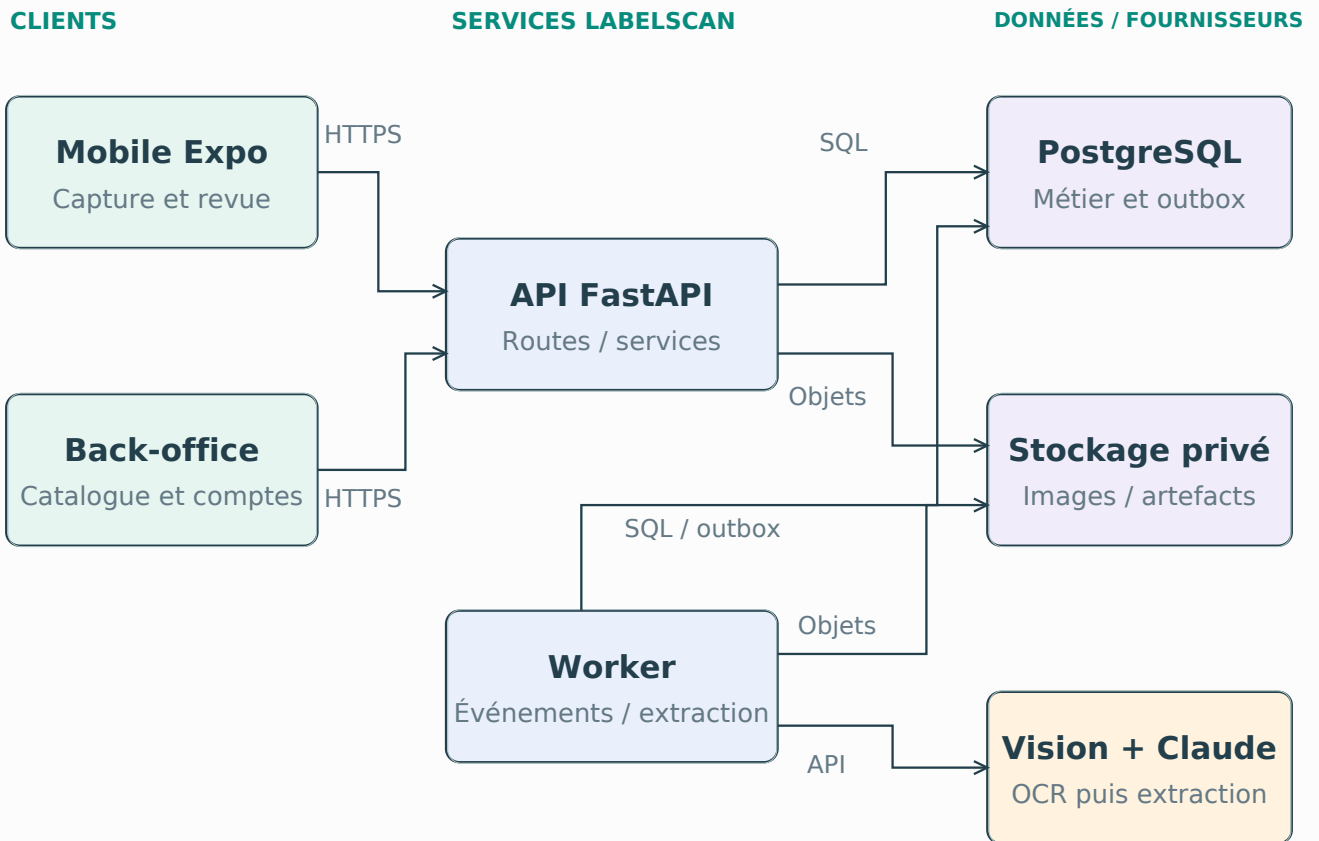
La session navigateur utilise un token d'accès conservé en mémoire et un cookie de renouvellement géré par le serveur. La navigation suit l'identité résolue ; les demandes de données sont vérifiées par l'API.

Les composants d'interface comprennent des sélecteurs, un calendrier de plage de dates, des filtres et une visionneuse de photo. L'application dispose de tests de composants et de scénarios de parcours Playwright.

Références : S04, S09, S12, S14.

07 - Vue des composants

Le serveur forme un monolithe modulaire, exécuté dans des processus API et worker distincts. Les applications partagent la même API et le même catalogue métier.



Traits pleins : connexions d'exécution. L'outbox réside dans PostgreSQL.

Les traits pleins représentent les échanges d'exécution. Les services OCR et LLM sont appelés par le worker. PostgreSQL conserve les données structurées ; le stockage privé conserve les objets image et les artefacts de traitement.

Le backend construit et sert également les ressources du back-office. L'outbox est une table PostgreSQL reliant les transactions métier aux traitements asynchrones.

Références : S06, S07, S10, S11.

08 - Les modules du serveur

Le code Python est organisé en contextes métier. La couche d'assemblage sélectionne les adapters, construit les services et enregistre les routes et consommateurs.

Contexte	Responsabilité
identity	Comptes, sessions, magasins, portails et affectations
ingestion	Réception de l'image, extraction, corrections et revue
traceability	Enregistrement des lots et lecture du catalogue
haccp	Plans de contrôle, relevés et cycle des alertes
audit	Frontière de l'historique d'audit
compliance	Frontière réservée dans l'organisation modulaire
platform	Accès base, HTTP, stockage, outbox et observabilité

À l'intérieur d'un contexte, les dépendances suivent adapters → application → domain. Le domaine exprime les états et règles ; l'application coordonne les ports ; les adapters réalisent les échanges SQL, HTTP, OCR et LLM.

Les workflows transversaux s'appuient sur les identifiants métier et les événements. Le relais outbox et les consommateurs assurent le passage de l'ingestion à la traçabilité, puis aux alertes.

L'architecture concentre la modélisation sur les comportements qui portent des invariants : capture, extraction, revue et cycle d'alerte. Les modules d'annuaire et de lecture restent plus simples. Import Linter vérifie les frontières de dépendances enregistrées dans le dépôt.

Les évolutions de données sont exprimées par les migrations Alembic. Les versions de profil, de prompt et de règle accompagnent les enregistrements concernés.

Références : S06, S07, S10, S14.

09 - Du cliché à l'extraction

La requête d'ingestion reçoit les octets de l'image et le contexte autorisé. Elle valide le contenu, calcule son checksum et écrit le fichier via le port de stockage. La transaction SQL enregistre la capture, la référence d'artefact et l'événement ingestion.raw_stored.

La réponse HTTP 202 est produite après le commit. Le traitement OCR et LLM est pris en charge par le worker.

Phase	Traitement	Donnée produite
GS1	Décodage et validation des identifiants d'application reconnus	Candidats déterministes et avertissements
OCR	Lecture de l'image par Google Vision	Texte, confiance moyenne et informations de page
Qualité	Évaluation de l'utilisabilité du texte	Décision de poursuite et aperçu intermédiaire possible
LLM	Extraction structurée selon le profil	Champs proposés, confiance, évidence et diagnostics
Gate	Vérification des preuves et règles	Champs validés, corrigés ou orientés vers la revue
Réconciliation	Application de la priorité GS1 et examen des conflits	Run finale associée à l'ingestion

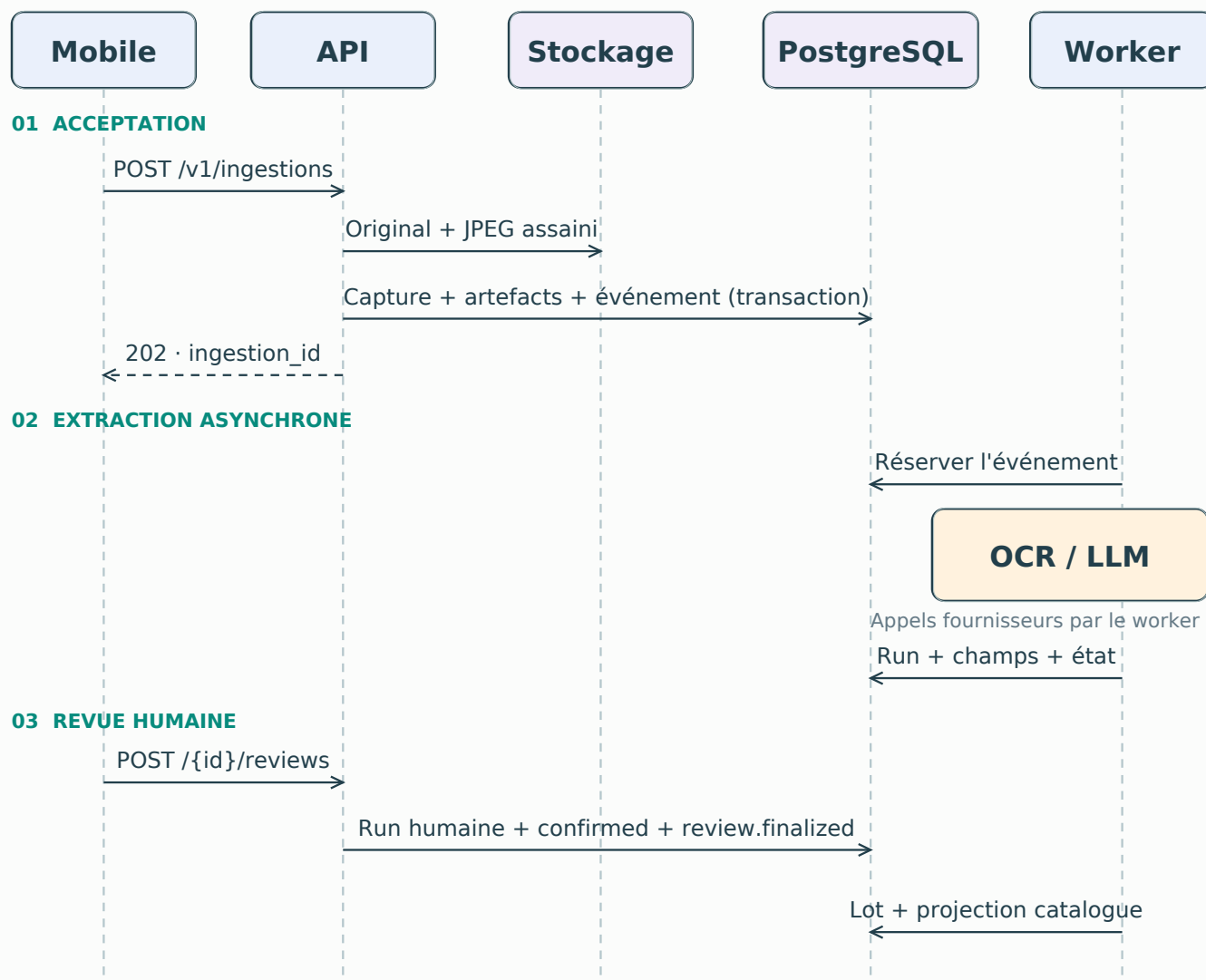
Les champs calculables à partir du code-barres sont traités avant le modèle. Leurs noms peuvent être transmis comme indication de travail déjà résolu. La réconciliation applique ensuite cette priorité dans le résultat.

Le worker conserve les artefacts OCR et LLM normalisés et les réutilise lorsqu'ils sont déjà enregistrés. Les tentatives fournisseur sont bornées ; un résultat inutilisable ou un échec de traitement possède un état explicite.

Références : S05, S06, S07.

10 - Capture et publication

Le diagramme distingue l'acceptation de la capture, l'extraction asynchrone, la revue humaine et l'enregistrement au catalogue.



La revue utilise `/v1/ingestions/{id}/reviews`. Pointillés : retour.

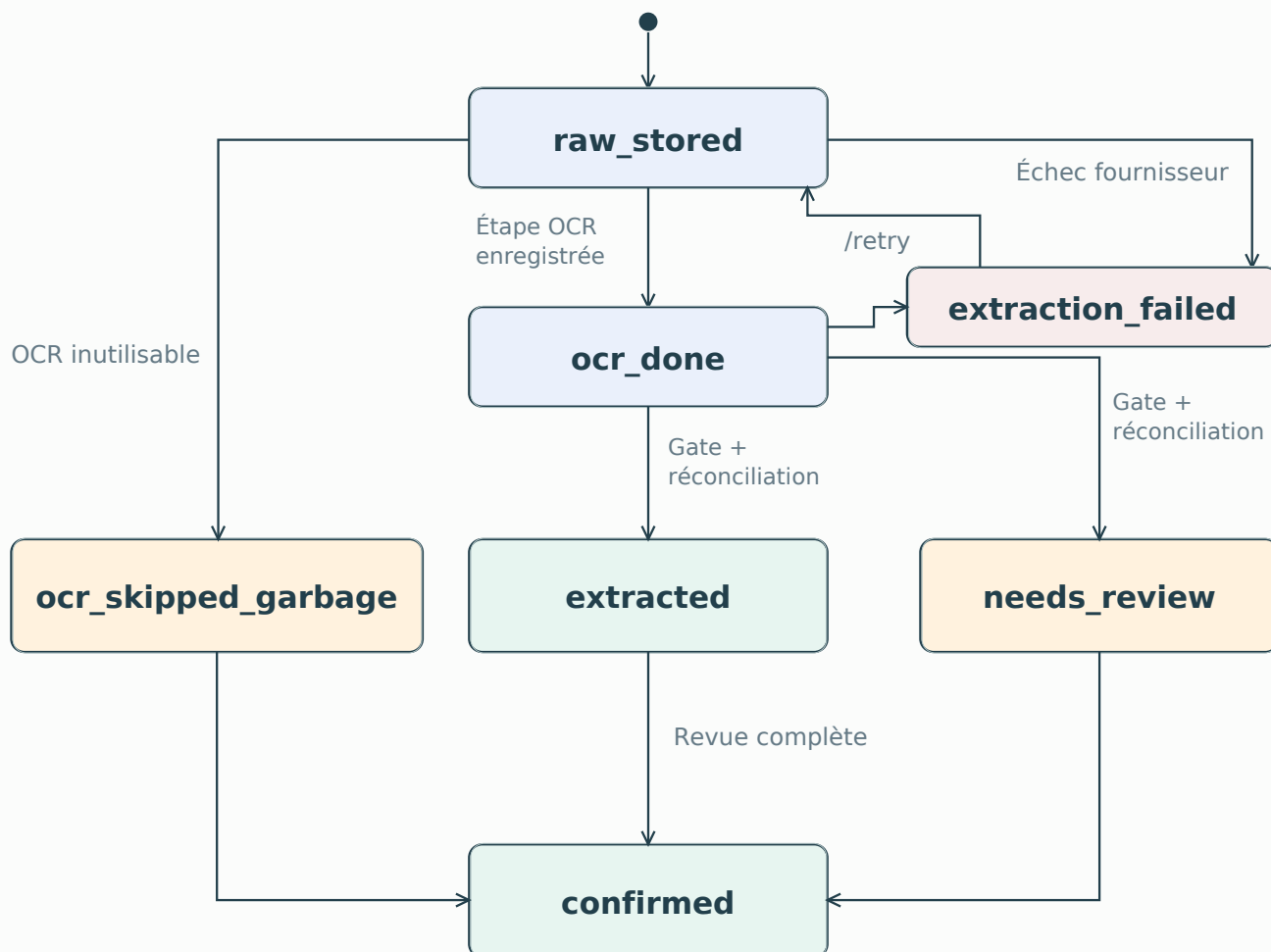
La validation humaine et l'événement `review.finalized` sont écrits ensemble. Le consommateur de traçabilité enregistre le lot et sa projection ; le mobile et le web retrouvent ensuite l'arrivage par les routes de consultation.

Une flèche en pointillés représente un retour. Les repères numérotés regroupent les trois phases du parcours ; ils rendent explicite la séparation entre acceptation HTTP et publication métier.

Références : S05, S07, S08.

11 · Les états du traitement

L'état de l'ingestion décrit l'avancement serveur. Le résultat de l'extraction et la carte de scan mobile portent des informations complémentaires.



Les états `extracted` / `needs_review` sont aussi accessibles sans étape `ocr_done`.

`ocr_done` identifie l'étape OCR enregistrée, avec ou sans champs d'aperçu. `extracted` et `needs_review` décrivent le résultat après gate et réconciliation ; la finalisation humaine conduit à `confirmed`. Le chemin `ocr_skipped_garbage` identifie un OCR jugé inutilisable. Une relance par `/retry` remet une extraction échouée à `raw_stored`.

Le mobile traduit le travail en états de carte : `submitting`, `extracting`, `ready`, `recapture_required`, `submit_error` et `extract_error`. Ces noms concernent le parcours local et ne remplacent pas les états serveur.

Le polling est borné, suspendu en arrière-plan et repris au retour de l'application. Les données d'extraction peuvent être relues depuis leur identifiant serveur.

Références : S03, S05, S07.

12 · Le contrat de l'IA

Le modèle reçoit le texte OCR, le profil de l'ingestion et les noms de champs déjà résolus. Le JSON retourné possède une propriété `fields`, contenant un sous-ensemble sans doublon des champs autorisés.

```
{
  "fields": [{
    "name": "commercial_designation",
    "value": "Saumon",
    "confidence": 0.92,
    "evidence": ["Saumon"],
    "validation_status": "present"
  }]
}
```

Cet exemple illustre uniquement le format. `name`, `value`, `confidence` et `evidence` sont les propriétés de base. `validation_status` et `warnings` complètent les diagnostics. Une absence ordinaire peut être omise ; un champ ambigu peut être émis avec une valeur nulle et ses diagnostics.

Les états acceptés sont `present`, `missing`, `ambiguous`, `normalized`, `unnormalizable` et `invalid`. Les valeurs métier sont des chaînes ou `null` ; les preuves sont des listes de fragments textuels.

Le serveur contrôle les noms, doublons, longueurs, tailles d'artefact, confiance et diagnostics. Le gate vérifie l'ancrage exact des preuves dans l'OCR. La confiance d'un champ ancré combine celle du modèle et de l'OCR en prenant la plus basse.

Le prompt détaillé Poissonnerie v2 porte l'identifiant `seafood-label-extraction/v3.3.0`. Les autres profils utilisent un prompt généré à partir de leur contrat. Les règles appliquées et la provenance du traitement sont enregistrées avec la run.

Références : S01, S06, S08.

13 · Provenance et versions

Chaque proposition possède une origine : traitement d'extraction OCR/LLM, décodage GS1 ou saisie humaine. Cette distinction accompagne les champs stockés et permet de lire l'historique de la capture. Le traitement d'extraction comprend aussi des règles OCR déterministes.

Donnée conservée	Utilité
Image originale et SHA-256	Relier la capture aux octets enregistrés
Image assainie	Fournir une représentation adaptée aux traitements et lectures
Artefact OCR normalisé	Conserver le texte et les métadonnées utilisées par l'extraction
Artefact LLM normalisé	Conserver les propositions structurées et leur provenance
Extraction run	Identifier tentative, versions et résultat du traitement
Champs et preuves	Expliquer chaque valeur proposée et son état
Run humaine	Conserver la version décidée par l'opérateur

Les runs et leurs champs sont ajoutés à l'historique. Une correction crée une nouvelle run et préserve les propositions antérieures. Les lectures utilisent les références de capture et de run pour retrouver les versions.

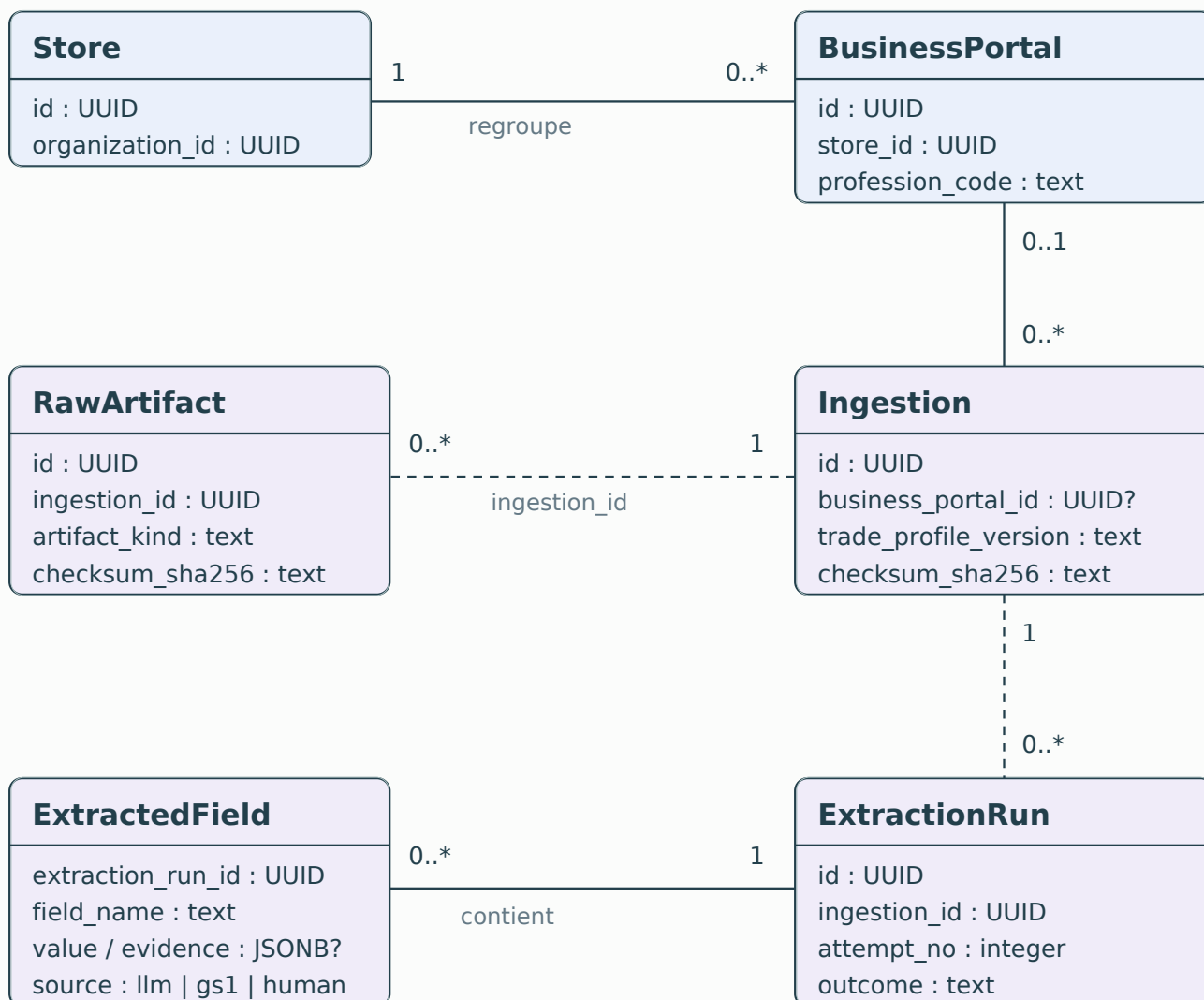
Les identifiants de corrélation et de trace relient les échanges et événements. L'audit SQL accompagne les mutations des tables qui lui sont rattachées, dans la transaction métier.

Le profil figé conserve le sens de la fiche. Les noms de modèle, de prompt et de règles identifient la configuration de traitement employée. La projection catalogue rassemble les données utiles à la consultation courante.

Références : S06, S08, S10.

14 · Modèle de données

La vue UML ci-dessous regroupe les relations qui relient capture, profil et historique. Les multiplicités indiquent le nombre de lignes associées ; les attributs sont volontairement limités aux identifiants et aux données structurantes.



Plein : clé étrangère. Pointillé : lien par identifiant. ? : valeur nullable.

Le portail appartient à un magasin et porte la profession. L'ingestion référence ce contexte et sa version de profil. Les artefacts et les runs appartiennent à l'ingestion ; les champs appartiennent à une run.

Les entités de catalogue, de session, d'affectation et d'alerte complètent ce noyau. Les migrations SQL décrivent leurs contraintes et relations complètes.

Références : S01, S08, S09, S10.

15 · Catalogue et traçabilité

Le consommateur de `review.finalized` vérifie le statut confirmé, la complétude du profil et la source humaine des champs. Il enregistre le lot, les références produit et fournisseur utiles, puis émet `batch.registered` ou `batch.flagged` selon le résultat des contrôles de cohérence.

La projection d'arrivage est une vue de lecture. Elle rassemble les valeurs du profil, le lot, le fournisseur, la date d'enregistrement, l'état et le périmètre de la capture. Elle est créée lors de l'enregistrement du lot issu de la revue complète.

Consultation	Fonction
Liste	Arrivages du périmètre résolu, paginés et triés
Recherche	Texte sur les champs projetés, lot et fournisseur
Filtres	Période, métier et critères exposés par le contrat
Détail	Champs du profil et photographie de l'arrivage
Traçabilité	Références de lot et lien avec la capture d'origine
Export	Fichier JSON ou CSV généré par le serveur, puis partagé depuis le mobile

Les requêtes de catalogue appliquent d'abord les critères d'organisation et de portail. Les index couvrent notamment les dates, les dimensions de périmètre et la recherche textuelle sur la projection `JSONB`.

Le contexte HACCP utilise le plan actif pour les alertes de péremption ; un événement `batch.flagged` produit directement une alerte de cohérence. Une alerte ouverte peut être acquittée puis résolue, ou résolue directement. Ses changements d'état sont audités.

Références : S04, S08, S10, S13.

16 · L'interface HTTP

Les applications utilisent l'API versionnée /v1. Les corps structurés sont en JSON ; la capture utilise un upload multipart. Les routes de lecture des images fournissent les octets correspondants au contexte autorisé.

Famille	Routes représentatives
Sessions web	/v1/auth/login, /v1/auth/refresh, /v1/auth/logout
Session d'organisation	/v1/o/{organization_slug}/auth/login
Sessions mobiles	/v1/mobile/auth/login, /refresh, /logout
Annuaire	/v1/stores, /v1/admins, /v1/managers
Ingestion	/v1/ingestions, /v1/ingestions/{ingestion_id}
Résultats	/v1/extraction-runs/{run_id}
Revue	/v1/ingestions/{ingestion_id}/reviews
Catalogue	/v1/professions, /v1/arrivals, /v1/arrivals/{batch_id}
Export	GET /v1/arrivals/export
Traçabilité	/v1/batches/{batch_id}, /v1/alerts
Plateforme	/v1/health/live, /v1/health/ready, /v1/version

La méthode HTTP précise l'opération. L'inventaire OpenAPI contient les schémas, paramètres et réponses des routes implémentées.

Les réponses d'erreur utilisent application/problem+json avec un code stable `error_code`, un statut HTTP, une description et un indicateur retrievable. La corrélation et la trace relie une réponse au traitement associé.

La clé Idempotency-Key identifie une mutation jouable lorsqu'elle est requise par la route. Le même identifiant d'opération est conservé pendant les retries du client.

Références : S05, S09, S13, S15.

17 - Capture et revue par API

POST /v1/ingestions accepte une image multipart et son contexte de capture. L'identité authentifiée détermine le périmètre métier. La réponse 202 retourne l'identifiant d'ingestion pour suivre le traitement.

La lecture GET /v1/ingestions/{ingestion_id} expose l'état, les artefacts et les résultats disponibles. GET /v1/extraction-runs/{run_id} ouvre une tentative et ses champs, avec leur confiance, leur source et leur évidence.

Opération	Effet
Soumettre l'image	Enregistrer la capture et programmer l'extraction
Lire l'ingestion	Consulter l'avancement et les références du traitement
Lire une run	Consulter les champs d'une tentative
Finaliser la revue	Enregistrer une version humaine complète et l'événement associé

La finalisation utilise POST /v1/ingestions/{ingestion_id}/reviews. La structure fields associe chaque nom de champ à sa valeur finale. Le serveur attend le profil complet, avec des chaînes non vides. Le formulaire mobile applique les règles de valeurs obligatoires décrites à la section 05.

Après validation des valeurs, une transaction vérifie le statut et le profil de l'ingestion verrouillée, ajoute la run humaine et ses champs, enregistre la rotation, confirme l'ingestion et écrit review.finalized. Le résultat d'idempotence est conservé avec cette opération.

Les routes de correction unitaire et de confirmation historique existent également dans l'inventaire HTTP. Le parcours mobile de revue complète s'appuie sur /reviews, qui porte la finalisation et la publication asynchrone décrites ici.

Références : S05, S08, S15.

18 - Consultation et annuaire

GET /v1/arrivals est l'entrée catalogue. Il combine le périmètre autorisé avec les paramètres de recherche, filtres, tri et pagination. Le détail d'un arrivage est adressé par l'identifiant de lot ; sa photographie se consulte par la route image associée.

La liste des professions décrit les profils disponibles. Les clients utilisent leurs définitions métier pour présenter les groupes de champs, formulaires et détails dans un ordre cohérent.

Ressource	Identité et usage
Organisation	Regroupe les magasins et comptes du périmètre
Magasin	Représente l'établissement physique
Portail métier	Associe organisation, magasin et profession
Affectation	Relie un utilisateur à son portail
Compte	Porte le rôle et les informations de connexion
Arrivage	Conserve les dimensions du contexte lors de son enregistrement

Les opérations d'annuaire permettent de gérer les magasins et les comptes accessibles au rôle courant. Les portails et affectations portent un état actif. Le contexte courant du manager est résolu à partir de son affectation.

Le changement d'affectation agit sur les accès de session. Les snapshots historiques d'organisation, magasin, portail, métier et acteur restent attachés aux captures et arrivages déjà créés.

Le catalogue offre ainsi une vue actuelle des fiches tout en conservant leur contexte d'origine. Les lectures de lot et d'alerte complètent cette vue pour les fonctions de traçabilité.

Références : S04, S08, S09, S10, S15.

19 - La plateforme d'exécution

Le dépôt réunit les sources mobile, web et serveur, ainsi que les fichiers de construction et de déploiement. L'image backend embarque l'API Python et les ressources web compilées.

Élément	Configuration du dépôt
Mobile	Expo SDK 57 ; React Native 0.86 ; React 19.2
Web	React 19, Vite et Wouter
Serveur	Python 3.11+ ; image de production Python 3.13 ; FastAPI
Base	PostgreSQL 16 ; migrations Alembic
Local	Docker Compose : base, migrations, seed, API et quatre workers
Managed	API/worker avec base externe et stockage S3-compatible
Single-VPS	Composition avec PostgreSQL et volume privé d'images

Le port de stockage permet aux mêmes services métier d'utiliser un répertoire privé ou un stockage objet. Les paramètres d'environnement sélectionnent la topologie et les adapters.

La configuration distingue le développement et les profils de build mobile. L'adresse d'API est intégrée au bundle par EXPO_PUBLIC_API_BASE_URL. Les fichiers .env.example décrivent les variables locales et serveur ; les fichiers Compose décrivent leurs dépendances d'exécution.

La démo locale contient neuf arrivages fondés sur des photographies d'étiquettes. Son manifeste versionné relie les images, leurs checksums et les observations de champs. Le seed peuple les comptes et exemples depuis les données de démonstration.

Références : S11, S12, S16.

20 · Traitements et observabilité

Le relais outbox sélectionne les événements associés à ses consommateurs enregistrés. Il réclame une ligne éligible avec FOR UPDATE SKIP LOCKED, exécute le traitement et enregistre le marqueur de consommation.

Mécanisme	Fonction
Outbox transactionnelle	Conserver les événements produits avec les écritures métier
Marqueur (consumer, event_id)	Reconnaître les traitements déjà enregistrés
Backoff	Espacer les nouvelles tentatives après erreur
Dead letter	Identifier les événements ayant épuisé leur budget de reprise
Artefacts normalisés	Réutiliser les résultats fournisseur déjà persistés
Corrélation et trace	Relier requêtes, événements et traitements

Les appels externes sont mesurés à la frontière des adapters. Les événements external_api_metrics regroupent volumes, succès, erreurs, limitations, appels en cours et latences. La configuration porte les limites d'appels par worker pour Vision et Anthropic.

Le traitement LLM expose également les compteurs de tokens, de création et de lecture de cache ainsi que les identifiants de modèle et de prompt. Le cache du préfixe statique Poissonnerie est conditionné par la mesure du nombre de tokens et les capacités du modèle configuré.

Les endpoints health/live, health/ready et version décrivent respectivement le processus, l'accessibilité de PostgreSQL et du stockage d'objets, et l'identité de version. Les journaux structurés et les états persistés offrent des vues complémentaires de l'exécution.

Références : S06, S07, S11, S13.

21 · Qualité logicielle

La vérification du dépôt couvre les règles métier, les adapters, les interfaces et la persistance. Les tests utilisent des doubles de fournisseurs pour reproduire les sorties, erreurs et reprises du traitement.

Couche	Couverture présente
Mobile · Jest	Capture, orientation, états de scan, session, outbox, revue et exports
Web · Vitest	Routes, périmètres métier, composants, filtres et détails
Web · Playwright	Parcours d'accès aux portails et interactions de navigation
Backend · Pytest	Profils, OCR, contrat LLM, GS1, évidence, revue, catalogue et HTTP
PostgreSQL	Migrations, contraintes, append-only, audit, transactions et périmètres
Structure	Typage TypeScript, lint Python, frontières d'import et inventaire OpenAPI

Les scénarios de reprise reproduisent notamment la répétition d'une requête, le changement d'identité, les erreurs réseau et la réutilisation d'une clé d'idempotence. Les scénarios d'extraction distinguent une valeur ancrée dans le texte, une valeur sans preuve et un conflit avec le code-barres.

Les tests de revue vérifient que le profil est complet et que les valeurs humaines sont associées à une nouvelle run. Les tests de publication suivent l'événement jusqu'au lot et à la projection catalogue.

Le manifeste de démonstration conserve les observations présentes, absentes ou ambiguës par image. Il relie les valeurs à une évidence et à un checksum d'image, ce qui rend les exemples reproductibles.

Les scripts de contrôle et les workflows CI sont versionnés avec l'application. Ils constituent les points d'entrée exécutables de cette couverture.

Références : S03, S14, S16.

22 - Références du dépôt

Les chemins ci-dessous identifient les sources de cette synthèse. Les références couvrent les composants décrits et donnent accès aux schémas détaillés.

Réf.	Source	Contenu
S01	Profils serveur	Champs, formats et versions
S02	Écrans et services mobile	Capture, revue, session et catalogue
S03	Tests mobile	Parcours et invariants locaux
S04	Application web	Portails, routes et arrivages
S05	HTTP ingestion	Capture, lecture et revue
S06	Adapters ingestion	OCR, LLM et orchestration
S07	Assemblage et outbox	Processus et événements
S08	Traçabilité	Lots et projections
S09	Identité	Comptes, magasins et portails
S10	Migrations	Modèle relationnel et contraintes
S11	Compose local, déploiement	Topologies d'exécution
S12	Mobile, web, serveur	Versions et scripts
S13	Plateforme HTTP et HACCP	Lectures transversales et alertes
S14	Tests serveur et CI	Contrôles automatisés
S15	OpenAPI	Inventaire HTTP généré
S16	Manifeste démo et seed	Exemples photographiques

Le PDF et les aperçus PNG sont générés à partir des sources Markdown et des définitions de diagrammes par `scripts/build_documentation.py`. Le fichier Excalidraw rassemble les quatre schémas sur un même canevas éditable.

G01 - Organisation du dépôt

LabelScan est un dépôt commun à trois applications : le mobile, le back-office et le serveur. Chaque application conserve ses dépendances et ses tests. Les documents relient ces composants par un parcours fonctionnel partagé.

Labelscan/	
App.tsx, src/	Application mobile React Native
android/, ios/	Projets natifs generes localement par Expo
web/	Back-office React et tests navigateur
server/	
src/labelscan/	API, cas d'usage et traitements
migrations/	Evolutions du schema PostgreSQL
tests/, demo/	Verification et donnees de demonstration
deploy/	Compose, proxy et scripts de livraison
scripts/	Controles transverses et generation PDF
.github/	Integration continue et automatisation
docs/	Guides, UML et contrat OpenAPI
output/pdf/	Edition paginee

Trois points d'entrée

Application	Premier fichier à lire	Ce qu'il assemble
Mobile	RootNavigator.tsx	Liste, caméra, revue et détail produit
Web	AppRouter.tsx	Connexion, portails, arrivages et espaces administratifs
Serveur	http_app.py	Routes, middlewares et fichiers du back-office

Les profils métier ont un contrat canonique côté [serveur](#). Les clients en présentent les libellés et regroupements adaptés à leur interface.

Les répertoires [assets](#) et [vendor](#) contiennent respectivement les ressources de l'application et une dépendance locale. Les fichiers package-lock.json et les verrous Python fixent les dépendances résolues.

G02 - Environnement local

Le démarrage commun utilise Docker Compose : PostgreSQL, les migrations, le jeu de démonstration, l'API et les workers sont décrits dans [docker-compose.yml](#). Le back-office est compilé dans l'image serveur.

Préparer puis démarrer

Depuis un clone neuf, copier les modèles puis renseigner leurs valeurs locales selon la [fiche de configuration](#).

```
cp server/.env.example server/.env
cp server/demo/credentials.example.json \
  server/demo/credentials.local.json
docker compose --env-file server/.env up --build -d --wait
```

L'option `--env-file server/.env` fournit à Compose les variables utilisées dans le YAML. Le même fichier est chargé comme environnement des processus serveur. Le démarrage respecte les dépendances : base disponible, migrations terminées, démonstration chargée, puis services applicatifs.

Adresse locale	Usage
http://localhost:8000/backoffice/	Interface web intégrée
http://localhost:8000/docs	Exploration interactive de l'API en développement
http://localhost:8000/v1/health/live	Présence du processus API
http://localhost:8000/v1/health/ready	Accès à PostgreSQL et au stockage d'objets

Observer et arrêter

```
docker compose --env-file server/.env ps
docker compose --env-file server/.env logs --tail=50 server worker
docker compose --env-file server/.env stop
```

stop arrête les conteneurs sans supprimer les volumes. Après cet arrêt, `docker compose --env-file server/.env start db server worker` reprend les services existants sans rejouer le chargement de démonstration. Les arrivages préchargés permettent de parcourir le catalogue ; une nouvelle capture traverse le traitement OCR/IA configuré.

G03 - Développer les interfaces

Application mobile

Node.js 22.13+ dans la branche 22 et les dépendances installées par npm ci constituent l'environnement JavaScript. Copier `.env.example` dans `.env`, puis renseigner `EXPO_PUBLIC_API_BASE_URL` avec l'URL joignable depuis l'appareil.

Avec Android Studio, le SDK et un appareil ou émulateur configurés :

```
LABELSCAN_BUILD_PROFILE=development npm run android
```

Sur macOS, avec Xcode et son simulateur configurés :

```
npm run ios
```

Ces commandes construisent et lancent le client natif de développement. Pour une session suivante avec ce client installé :

```
LABELSCAN_BUILD_PROFILE=development npx expo start --dev-client
```

Le profil `development` active la configuration Android destinée à l'API locale HTTP. Les réglages natifs sont regroupés dans [app.config.js](#).

Back-office web

```
npm --prefix web ci  
npm --prefix web run build
```

Le résultat est `web/dist`. L'API peut servir ce répertoire avec `LABELSCAN_STATIC_DIR` ; le [guide serveur](#) donne la commande. Le client web utilise des URL `/v1/...` de même origine. Le serveur Vite seul sert le travail d'interface, sans proxy API configuré dans le dépôt.

Les [constantes graphiques mobile](#) centralisent couleurs, typographie et espacement. Le [style web](#) définit les adaptations d'écran et le focus visible. Les composants de sélection gèrent la navigation clavier, couverte par des tests ciblés.

G04 - Lire le serveur par couches

Le serveur est un **monolithe modulaire** : un même projet Python contient plusieurs domaines métier. L'API et les workers sont des processus distincts qui réutilisent ses composants.

Couche	Rôle	Exemple dans l'ingestion
domain	Types et règles métier	Normalisation et contrôle des champs
application	Enchaînement d'un cas d'usage	<code>finalize_review.py</code>
adapters	Entrées HTTP et implémentations techniques	Routes, repositories SQL, OCR et IA
app	Assemblage des composants	Fabrique FastAPI et lancement du worker
platform	Services partagés	Base, stockage, HTTP et observabilité

Un **port** est une interface attendue par un cas d'usage. Un **adaptateur** fournit son implémentation. La finalisation d'une revue dépend ainsi d'un repository, et non d'une requête SQL écrite directement dans la route HTTP.

Domaines actifs

- identity : sessions, organisations, magasins, utilisateurs et accès.
- ingestion : captures, extraction, champs et revue humaine.
- traceability : lots et projection de consultation du catalogue.
- haccp : règles d'alerte et cycle de traitement des alertes.

Le [contrat d'import](#) formalise les dépendances autorisées entre couches et domaines. `lint-imports` contrôle ces frontières ; les tests exercent les règles et les adaptateurs.

Le [guide serveur](#) regroupe les commandes d'installation, d'exécution et de test. La [vue des modules](#) complète cette lecture structurelle.

G05 - Suivre une fonctionnalité

La confirmation d'une fiche est un parcours vertical : une action d'interface traverse HTTP, un cas d'usage, une transaction et un traitement asynchrone.

Étape	Responsabilité	Source à lire
1. Saisie	Vérifier puis conserver la revue dans l'outbox mobile	ReviewScreen.tsx
2. Transport	Envoyer l'opération conservée à la route /reviews	api.ts
3. Entrée HTTP	Recevoir et convertir le corps de la requête	router.py
4. Cas d'usage	Contrôler le jeu de champs et les valeurs finales	finalize_review.py
5. Persistance	Résoudre le profil enregistré et écrire la revue avec son événement	sql_review_repository.py
6. Publication	Consommer l'événement puis alimenter le catalogue	Adaptateurs de traçabilité

Ce que le découpage explique

Le contrôle mobile aide l'opérateur pendant la saisie. Le contrôle serveur applique le contrat métier reçu par l'API. La transaction regroupe les écritures de la revue ; la publication intervient ensuite, via l'événement `review.finalized`.

L'**idempotence** reconnaît la répétition d'une même opération. Elle est distincte de la validation : une requête peut être correctement formée tout en répétant un travail déjà enregistré.

Le [diagramme de séquence](#) montre ces temps successifs. Le [test de finalisation](#) vérifie notamment la répétition sans doublon et la publication catalogue. Le [chapitre revue](#) décrit la complétude et les décisions NC permises.

G06 - Données et contrat HTTP

De la preuve à la consultation

Une ingestion identifie la capture. Ses artefacts désignent les fichiers conservés. Les runs regroupent les résultats successifs d'extraction ou de revue ; leurs champs portent les valeurs et leur provenance. Le catalogue est une projection de lecture destinée aux interfaces.

Élément	Rôle dans le modèle
Organisation, magasin, portail	Situer les utilisateurs et les données dans leur contexte métier
Ingestion et artefacts	Relier la capture, son état et ses fichiers
Run et champs extraits	Conserver les versions, valeurs, sources et évidences
Lot et projection catalogue	Alimenter la consultation d'un arrivage

Le [modèle UML](#) distingue les relations SQL déclarées des associations par identifiant. Les [migrations Alembic](#) décrivent l'évolution du schéma. Les colonnes JSONB stockent des valeurs structurées **dans PostgreSQL**, au sein du modèle relationnel.

Lire une route

Opération	Échange principal
POST /v1/ingestions	Photo multipart et contexte ; réponse 202 avec identifiant
GET /v1/ingestions/{id}	État de la capture et informations d'extraction
POST /v1/ingestions/{id}/reviews	Revue complète selon le profil enregistré
GET /v1/arrivals	Consultation filtrée et paginée du catalogue
GET /v1/arrivals/export	Export serveur JSON ou CSV

Le [contrat OpenAPI](#) décrit les paramètres et structures exacts. Les [chapitres API](#) donnent une lecture par usage. Une réponse 202 signifie que la capture a été acceptée, pas que l'extraction est déjà terminée.

G07 - Vérifier à plusieurs niveaux

Les tests associent règles isolées, composants d'interface, accès aux données et parcours navigateur. Chaque niveau répond à une question différente.

Niveau	Exemples du dépôt	Outil
Règles mobile	Dates, GS1, complétude et gestion des opérations	Jest
Interface web	Profils, filtres, accès et panneaux de détail	Vitest et Testing Library
Parcours navigateur	Navigation, clavier et adaptation aux écrans	Playwright, API simulée
Métier et persistance	Revue, idempotence, transactions et projections	pytest avec PostgreSQL
Architecture	Frontières d'import et contrat HTTP à jour	import-linter et export OpenAPI

Commandes clients, depuis la racine

```
npm run typecheck
npm test -- --runInBand
npm --prefix web run typecheck
npm --prefix web test
npm --prefix web run build
```

Parcours navigateur, depuis web/

Après npm ci, installer Chromium puis lancer les scénarios :

```
npx playwright install chromium
npm run test:e2e
```

La configuration construit le web, ouvre son aperçu et exécute les scénarios aux formats 1440 × 900, 1024 × 768 et 390 × 844. Les tests [portal-access](#) couvrent notamment les accès, le clavier et le débordement horizontal.

Les [instructions serveur](#) utilisent une base dédiée aux tests. Les workflows [clients](#) et [backend](#) versionnent l'ordre des contrôles et leurs environnements.

G08 - De la source à l'exécution

Le [Dockerfile serveur](#) réalise une construction en plusieurs étapes : compilation du web, installation des dépendances Python verrouillées, puis assemblage de l'image d'exécution. Le serveur livre ainsi l'API et les fichiers statiques du back-office.

Processus	Commande du point d'entrée	Fonction
Migrations	migrate	Appliquer les évolutions Alembic
API	api	Exposer HTTP et servir le back-office
Worker	worker	Traiter les événements et les extractions
Démonstration locale	demo	Charger le jeu d'arrivages photographiques

Construction locale de l'image

Depuis la racine du dépôt :

```
docker build -f server/Dockerfile -t labelscan:local .
```

Cette commande construit l'artefact ; elle ne crée pas une base et ne déploie pas de service. Compose fournit ensuite la topologie, les paramètres et les volumes de l'environnement d'exécution.

Configurations versionnées

Le profil **single-VPS** regroupe proxy, base, API et worker. Le profil **managed** référence PostgreSQL externe et un stockage compatible S3. Le [guide de déploiement](#) identifie les fichiers, les paramètres et le déroulement du workflow VPS.

La chaîne de livraison distingue validation du code et mise en production. Les workflows de CI exécutent leurs contrôles ; le workflow de livraison sélectionne une révision et lance le script distant. La version, la disponibilité HTTP et la préparation des dépendances sont exposées par les endpoints d'exploitation.

Les [chapitres plateforme et observabilité](#) expliquent le rôle des services, journaux et compteurs dans cette exécution.

G09 - Repères RNCP niveau 5

Le repère de lecture retenu est le titre **Développeur web et web mobile**, RNCP37674, niveau 5. Le référentiel distingue la réalisation des interfaces et celle des traitements serveur. Source : [fiche officielle France compétences](#).

La correspondance décrit les réalisations présentées par ce dépôt. Elle relie une explication technique à des éléments observables dans le projet.

Axe de lecture	Réalisation LabelScan	Preuve à présenter
Environnement	Dépendances, services et configuration locale	Démarrage , verrous et Compose
Conception d'interface	Parcours opérateur, hiérarchie des écrans et profils	Parcours fonctionnel , navigation et constantes graphiques
Interface et adaptation	Composants React, styles responsive et clavier	Styles web , tests navigateur
Interface dynamique	Session, formulaires, appels API et suivi d'extraction	Revue mobile , client web
Persistance relationnelle	Modèle, migrations, SQL paramétré et transactions	Modèle UML , repository
Traitements métier	Validation, provenance et publication asynchrone	Parcours vertical , test de revue
Livraison	Image, services et workflow versionné	Déploiement , Dockerfile et CI

Fil conducteur d'une présentation

Partir du besoin de réception, suivre une capture jusqu'à sa fiche, puis expliquer une décision de conception à partir du code : séparation API/worker, transaction de revue ou adaptation des champs au métier. Terminer par les tests qui vérifient ce comportement et par les éléments qui composent la livraison.

Ce parcours relie **besoin, interface, traitement, données et vérification**, sans séparer la démonstration produit de son explication technique.

G10 - Lexique et conventions

Ingestion

Capture reçue par le serveur, identifiée et suivie par un état

OCR

Lecture du texte contenu dans l'image

Run

Version d'un résultat d'extraction ou d'une revue humaine

Profil métier

Ensemble versionné de champs et règles pour une profession

Revue

Intervention humaine qui renseigne et confirme les valeurs de la fiche

Outbox

File d'opérations mobile ou table d'événements serveur à traiter

Projection

Vue de lecture construite à partir des données et événements métier

Artefact

Fichier conservé et rattaché à une capture, par exemple une photographie

Extraction

Transformation des indices disponibles en champs du profil métier

Évidence

Extrait de texte associé à une proposition de champ

Portail

Contexte de travail associant magasin et métier

NC

Valeur non renseignée ; décision humaine ou règle d'affichage selon le contexte

Idempotence

Reconnaissance de la répétition d'une même opération

Arrivage

Fiche consultable d'un lot issu d'une capture revue

Vocabulaire technique

Repository : composant d'accès aux données. **Middleware** : traitement qui entoure le passage d'une requête HTTP.

CI : intégration continue, qui exécute automatiquement les contrôles du projet. **JWT** : format de jeton signé utilisé pour la session.

UML : notation de modélisation des structures et échanges. **GS1** : standards d'identification et de codage des informations produit.